Browser lifetime + stop rule: branch diff summary

- Branch: `fix/browser-lifetime-and-stop-rule` (pushed to `origin`)
- Base: `b676c2d`, the head of `fix/worker-isolation`, which records the Railway Trial 1 result
- **Release candidate (code + tests): `caaa191`.** Product code is identical to `f2288ce`; `caaa191` only changes the repro harness and fixture. The commit that adds these docs and the patch sits on top of it.
- `fix-browser-lifetime-and-stop-rule.patch` = `git format-patch b676c2d..caaa191 --stdout`, 4 commits. A patch cannot contain the commit that adds it.
- No merge, no deploy, no settings/.env change, no paid run. Scoring rules and BM25 are unchanged.

Feature gates (both off by default)

Change	Active when	Default production behaviour
Close+reap the child's Chromium before returning the browser permit	`WORKER_ISOLATION_ENABLED=true`, in isolated job children only (`isIsolatedChild()`)	Inert. The in-process `releaseBrowserSlot()` path is unchanged, apart from now returning a resolved Promise that callers `await`.
Batch stop rule S4	`BATCH_STOP_RULE_FAILED_JOBS` = N > 0 (both the isolated and in-process worker paths)	Inert. If the variable is unset, 0 or invalid, the retry path is the legacy one; test 9 checks this.

Commits since base

- `caaa191` test(browser-lifetime): per-document marker extraction, no-skip check, admission wait / permit hold / wall time in repro summary
- `f2288ce` **fix(worker-isolation): close+reap the child's Chromium before returning the browser permit; opt-in batch stop rule S4**
- `f3825cf` evidence(browser-lifetime): before/after repro runs (unlimited + TasksMax=400) with corrected per-profile browser counter (superseded by `evidence/browser-lifetime-candidate/`)
- `979db79` test(browser-lifetime): repro harness + fixture; count one browser per distinct puppeteer profile

Code/test files changed (`git diff --stat b676c2d caaa191`, excluding deliverables/)

```
server/lib/__fixtures__/browser-close-order-child.ts |  40 +++
server/lib/__fixtures__/browser-lifetime-child.ts    |  43 +++
server/lib/__fixtures__/marker-pdf.ts                |  27 +++
server/lib/batch-stop-rule.test.ts                   | 175 +++++++
server/lib/batch-stop-rule.ts                        | 137 +++++++
server/lib/browser-close-order.test.ts               | 150 +++++++
server/lib/processor.ts                              | 144 ++++++-
server/scripts/browser-lifetime-repro.ts             | 232 +++++++++
server/worker.ts                                     |  34 +
9 files changed, 970 insertions(+), 12 deletions(-)
```

The only product files changed are `processor.ts`, `worker.ts` and the new `batch-stop-rule.ts`. Everything else is tests, fixtures or the repro script. The full diff including the old evidence JSON is 14 files, +51,086/−12.

What changed

server/lib/processor.ts: browser lifetime in isolated children

Defect. In an isolated child, the browser permit, which the parent brokers per replica up to `MAX_CONCURRENT_BROWSER`, went back to the pool after every fetch. The child's Chromium stayed alive until the child exited, minutes later. So the number of live browsers per replica was bounded by the child count, not by the permit count. Locally that gave 4 live browsers with 1 permit.

Fix

- `releaseBrowserSlot()` now returns a Promise. In an isolated child it calls `releaseChildBrowserPermit()`. When the child releases its **last** held permit, that function does three things in order:
  1. It detaches every Chromium the child launched (`childBrowsers`, `sharedBrowser = null`).
  2. It closes each one (`close()` with a 5 s timeout) and confirms the process group is gone by scanning `/proc` for non-zombie members of the pgid. If the group survives a 5 s window, it sends SIGKILL to the group and checks again.
  3. Only then does it call `remoteRelease("browser")`.
- If there is no pid to probe, only a close that completed counts as confirmed.
- **Unconfirmed cleanup.** The permit is **withheld**, not returned, and `childBrowserRetireFailed` opens the browser circuit for that child, so it cannot deadlock itself. The parent reaps the process when the child exits.
- All three callers await the release, on the normal, refused-after-acquire and finally paths: `fetchWithBrowser`, `fetchPdfViaBrowser` and `fetchIssuerPdfsWithPrimedSession`. `fetchIssuerPdfsWithPrimedSession` also returns early if the circuit is open.
- **Reuse check.** `getSharedBrowser()` in an isolated child uses `browser.connected`. puppeteer-core 25.x has no `isConnected()`, so the old check always failed. The non-isolated check is unchanged.
- `[BROWSER-LIFECYCLE]` log lines record acquire, launch, close, reap and release.

server/lib/batch-stop-rule.ts and server/worker.ts: stop rule S4

Rule. Validation plan §7 rule S4: cancel the batch once ≥ N jobs have failed, or as soon as any job exhausts its retries.

Counting. Each job is counted at its **first** recorded failed attempt. `failed_jobs` only moves at retry exhaustion, which is why batch 1276 let 13 jobs burn 3 attempts each.

Where it runs. In `worker.ts`, on both the failed-result branch and the exception branch, before the retry re-enqueue. When the rule trips, it runs the `POST /batch/cancel` sequence in this order: flag, queue purge, snapshot, PG cancel. The caller then skips the retry.

Edge cases

- Cancellations and provider-quota pauses are not counted.
- If counting errors, the rule fails open to the legacy retry path.
- Purge and snapshot errors are not fatal.

Review focus

1. `processor.ts` `releaseChildBrowserPermit()` / `retireChildBrowser()`: the ordering, the withheld-permit path and the no-pid case.
2. `worker.ts` `stopRuleTripped()`: where it sits relative to `batchTerminal` and the retry branch.
3. `COUNT_BATCH_FAILURES_SQL`: not yet run against a real PG in this session (optional test skipped). See `TEST_RESULTS.md` §6.